Ensimag – Grenoble INP – UGA
Statistical Learning with Applications
Pedro L. C. Rodrigues
Alexandre Wendling

Year 2024-2025

pedro.rodrigues@inria.fr
alexandre.wendling@univ-grenoble-alpes.fr

Final **practical** exam – Duration: 3h

This exam is composed of four parts.

The maximum final grade is 20.0 points.

Be sure to **read through all of the questions** before starting to solve the exam.

All handwritten documents are allowed, as well as the electronic documents on your computer. No calculator or mobile phone allowed.

No internet connection is available from your computer, except to the following webpage :

- The `sklearn` documentation at : https://scikit-learn.org/stable/

This practical exam will be entirely done using Jupyter notebooks. Open each `ipynb` file located in the `$HOME/exam/` directory. Answer the questions directly in each file. The different `ipynb` files correspond to each of the parts of the exam.

Any answer not contained in the `ipynb` files will not be taken into account, unless explicitly mentioned by you in the `ipynb` file (for example you can state: please see figure `myfig.png`).

The contents of the `$HOME/exam/` directory are

- `exam.pdf` (this document)
- `Part1.ipynb`, `Part2.ipynb`, `Part3.ipynb`, `Part4.ipynb` (the files you have to complete and save)
- `/references/article-newman.pdf` (the PDF of the article useful in Part 4)
- `/references/slides/` (the PDFs of the slides of all classes)
- `/datasets` (the `csv` files to be used during the exam)

> You should regularly **save the current state** of your whole home directory using the icons on your desktop. To do that: (1) Save the files currently open in your editor then (2) Save your current work and continue the exam. **Before leaving the room** save your current work and stop the session. Please note that the backup operation is based on `rsync`, and does not store every intermediate saved version (in this order!). If your PC crashes, your latest backup can be recovered with the help of the teaching staff.

Good luck!

▶ Part 1: Multiple choice questions (0.5 point per question, 6 points in total)

For each question, write the letter in your `Part1.ipynb` file corresponding to the correct answer.

Each question has **exactly one correct** answer.

– Question 1

Given d-dimensional data $\mathbf{x}_{i=1}^{N}$, you run principle component analysis and pick P principle components. Can you always reconstruct any data point $\mathbf{x}_i$ for $i \in \{1...N\}$ from the d principle components with zero reconstruction error?

(A) Yes, if $P < d$.

(B) Yes, if $P = d$.

(C) Yes, if $P < N$.

(D) No, always.

– Question 2

Which of the following transformations to a data matrix $\mathbf{X}$ will affect the principal components obtained through PCA?

(A) Adding a constant value to all elements of $\mathbf{X}$.

(B) Multiplying one of the features of $\mathbf{X}$ by a constant.

(C) Adding an extra feature to $\mathbf{X}$ (i.e. an extra column) that is constant across all data points.

(D) None of the above answers.

– Question 3

In linear regression, we model $p(y \mid \mathbf{x}) \sim \mathcal{N}(\beta^\top \mathbf{x} + \beta_0, \sigma^2)$. The irreducible error in this model is

(A) σ^2

(B) $\mathbb{E}[y \mid \mathbf{x}]$

(C) $\mathbb{E}[(y - \mathbb{E}[y \mid \mathbf{x}])^2 \mid \mathbf{x}]$

(D) None of the above.

– Question 4

The decision boundaries of a logistic regression model:

(A) split classes using only one of the input features

(B) split classes using a combination of the input features

(C) often have curved shapes

(D) None of the above

– Question 5

How does the bias-variance decomposition of a ridge regression estimator compare with that of ordinary least squares regression?

(A) Ridge has larger bias, larger variance.

(B) Ridge has larger bias, smaller variance.

(C) Ridge has smaller bias, larger variance.

(D) Ridge has smaller bias, smaller variance.

– **Question 6**

Suppose you want to split a graph G into two subgraphs. Let L be G's Laplacian matrix. Which of the following could help you find a good split?

(A) The eigenvector corresponding to the second-largest eigenvalue of L

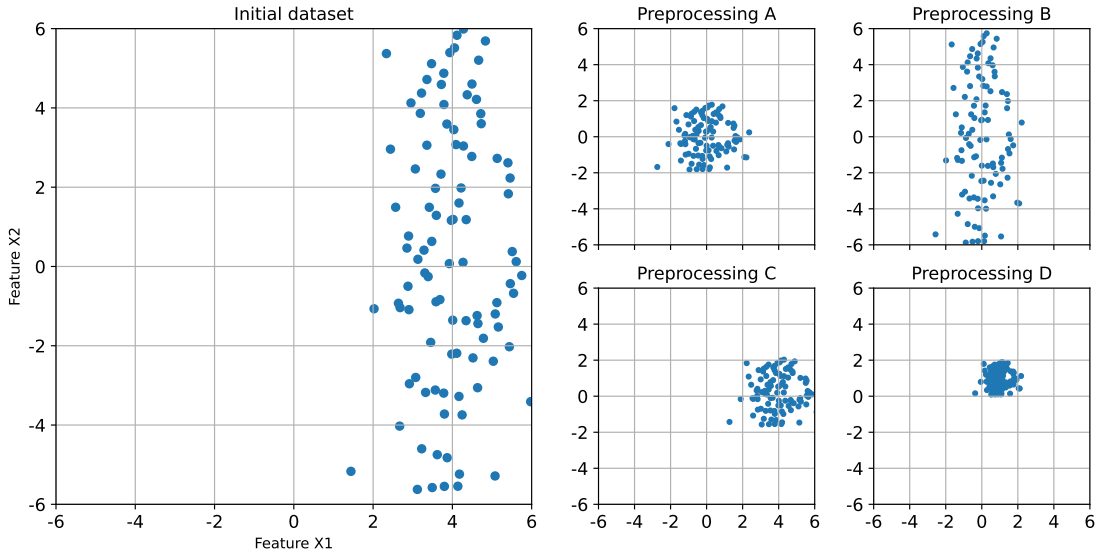
(B) The eigenvector corresponding to the second-smallest eigenvalue of L

(C) The left singular vector corresponding to the second-largest singular value of L

(D) None of the above

– **Question 7**

We define a 2-dimensional dataset represented graphically as in the left part of the figure below. If we process the dataset using a `StandardScaler` from `sklearn` with its default parameters, which of the results presented in the right part of the figure do you expect:



(A) Preprocessing A

(B) Preprocessing B

(C) Preprocessing C

(D) Preprocessing D

– **Question 8**

Why does random forests select a random subset of predictors when building each of its decision trees?

(A) To facilitate the cross-validation procedure afterwards

(B) To reduce bias of the final ensemble estimator

(C) To maximize the entropy of the decision trees

(D) To reduce the correlation between the decision trees in the ensemble

– **Question 9**

Which of the following statements is true about nearest neighbor classifiers

(A) Nearest neighbors can be slow to find in high-dimensional spaces

(B) Nearest neighbor classifiers can only work with the Euclidean distance

(C) Nearest neighbor classifiers do not need to store the training data

(D) None of the above

Observation: In fact the high-dimensional space problem is not really a matter of slowness but of inexistence of "nearest" neighbours per se. In fact, we would need an infinity amount of data to kNN work in high dimensions.

– **Question 10**

Suppose you have data with lots of outliers. Everything else being equal, and assuming that you do not do any pre-processing, what cost function will be less effected by these outliers?

(A) $(y - f(x))^2$

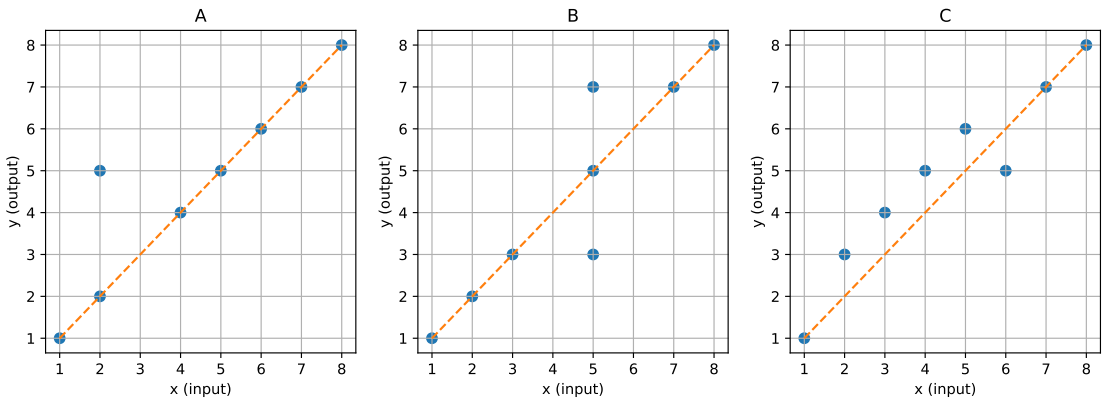
(B) $|y - f(x)|$

(C) $1/y * (y - f(x))^2$

(D) $(\log(y) - f(x))^2$

– **Question 11**

The figure below shows three different datasets (A, B, C) and the predictions of the same linear model (in orange) used to predict each of these datasets.



Choose the correct ordering with respect to the MSE (Mean Squared Error) loss on each dataset

(A) $\text{MSE}_B > \text{MSE}_A > \text{MSE}_C$

(B) $\text{MSE}_B > \text{MSE}_C > \text{MSE}_A$

(C) At least two of the MSE losses are equal

(D) $\text{MSE}_A > \text{MSE}_B > \text{MSE}_C$

– **Question 12**

Considering the same figure from the previous question, choose the correct ordering with respect to the MAE (Mean Absolute Error) loss on each dataset

(A) $\text{MAE}_A > \text{MAE}_C > \text{MAE}_B$

(B) $\text{MAE}_C > \text{MAE}_B > \text{MAE}_A$

(C) At least two of MAE losses are equal

(D) $\text{MAE}_B > \text{MAE}_C > \text{MAE}_A$

▶ Part 2: Multiple linear regression (5.0 points)

In this exercise, you will perform multiple linear regression on simulated data under different conditions. To ensure reproducibility on your results, set the seed with `numpy.random.seed(0)` at the beginning of your script.

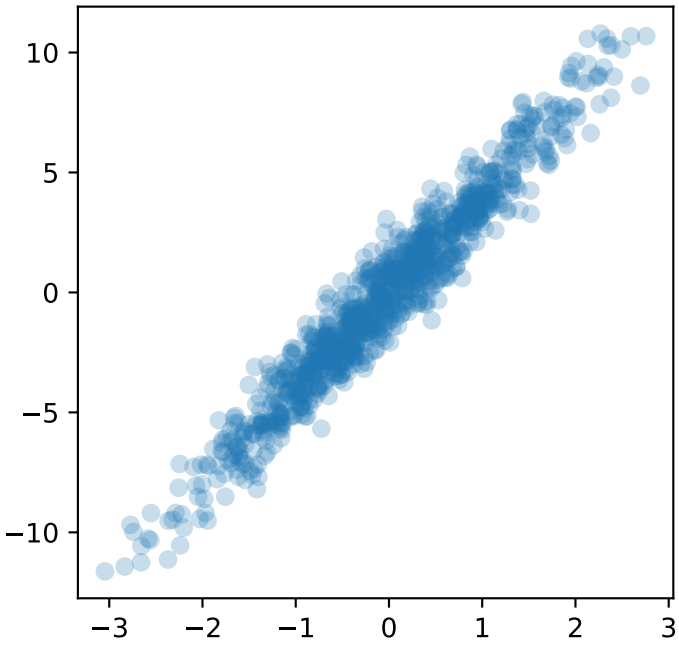
(a) Simulate a dataset of size $N = 1000$ of the following generating model:

$$\begin{aligned} X_{1,i} &= \varepsilon_{1,i} \\ X_{2,i} &= 4X_{1,i} + \varepsilon_{2,i} \\ Y_i &= X_{2,i} + X_{1,i} - 5 + \varepsilon_{3,i} \end{aligned}$$

where $i \in \{1, \ldots, N\}$ and the ε_{ij} are independent $\mathcal{N}(0, 1)$ random variables. For a given i, what is the distribution of $(X_{1,i}, X_{2,i})$? Plot the clouds of points of the simulated values of $(X_{1,i}, X_{2,i})_{i=1,\ldots,n}$. What is its shape? Can you write a closed-form expression for it? **(2.0 points)**

One possible code for plotting the samples would be:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

def generate_dataset(N, seed=0):
  np.random.seed(seed)
  X1 = np.random.randn(N)
  X2 = 4*X1 + np.random.randn(N)
  Y = X2 + X1 - 5 + np.random.randn(N)
  df = pd.DataFrame()
  df['X1'] = X1
  df['X2'] = X2
  df['Y'] = Y
  return df

df = generate_dataset(N=1000, seed=0)

fig, ax = plt.subplots(figsize=(4, 4))
ax.scatter(df['X1'], df['X2'], c='C0', alpha=0.25)
fig.show()
```



Since the samples are coming from a two-dimensional Normal distribution, their scatter plot has an a elliptical shape. The closed form expression for their joint pdf is:

$$\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) \quad \text{with} \quad \boldsymbol{\mu} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad \text{and} \quad \boldsymbol{\Sigma} = \begin{bmatrix} 1 & 4 \\ 4 & 17 \end{bmatrix}$$

To obtain the expression for the elliptical shape of the scatter points, one can check for the level curves of the pdf, i.e. the locus of points for which $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}) = k$ for different values of $k \in \mathbb{R}$. We get then different sets of $\boldsymbol{x}^\top = (x_1, x_2)^\top \in \mathbf{R}^2$ for which:

$$\frac{1}{Z} \exp\left(-\frac{1}{2}\boldsymbol{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{x}\right) = k \iff \boldsymbol{x}^\top \boldsymbol{\Sigma}^{-1} \boldsymbol{x} = \ell \iff 17x_1^2 - 8x_1x_2 + x^2 = \eta$$

for real constants k, ℓ, η.

Grading: 0.5 points for plot, 1.0 points for expression of Gaussian, 0.5 points for the expression of actual ellipse.

(b) Let us consider the following two regression models:

$$\begin{aligned} \text{Model A:} \quad Y_i &= \alpha_1 X_{1,i} + \alpha_0 + \tilde{\varepsilon}_{A,i} \\ \text{Model B:} \quad Y_i &= \beta_2 X_{2,i} + \beta_0 + \tilde{\varepsilon}_{B,i} \end{aligned}$$

where $\tilde{\varepsilon}_{A,i} \sim \mathcal{N}(0, \sigma_A^2)$ and $\tilde{\varepsilon}_{B,i} \sim \mathcal{N}(0, \sigma_B^2)$. What should be the values of $\hat{\alpha}_0, \hat{\alpha}_1, \hat{\sigma}_A^2, \hat{\beta}_0, \hat{\beta}_2, \hat{\sigma}_B^2$ when $N \to \infty$? Consider $N = 1000$ and check whether the estimates of the parameters are close to the true values that you've calculated. You can do your estimation any way you want, e.g. using the analytical formula for regression, functions from `sklearn`, functions from `statsmodels`, etc. Now do `np.random.seed(3)` and simulate again a dataset $X_{1,i}, X_{2,i}, Y_i$ for $n = 10$. Estimate the parameters. What happens? **(2.0 points)**

When $N \to \infty$, we should get for model A: $\alpha_1 = 5, \alpha_0 = -5, \sigma_A^2 = 2$. For model B we should get $\beta_2 = \frac{5}{4}, \beta_0 = -5, \sigma_B^2 = 1 + \frac{1}{16}$. Let's check this with `statsmodels`:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm

df = generate_dataset(N=1000, seed=0)
df['intercept'] = 1

model = sm.OLS(df['Y'], df[['intercept', 'X1']])
results_A = model.fit()
print('Model A')
```

```
## Model A
```

```
print(results_A.params)
```

```
## intercept   -5.040659
## X1           4.932665
## dtype: float64
```

```
print(results_A.mse_resid)
```

```
## 1.8457546652512453
```

```
model = sm.OLS(df['Y'], df[['intercept', 'X2']])
results_B = model.fit()
print('Model B')
```

```
## Model B
```

```
print(results_B.params)
```

```
## intercept   -5.058198
## X2           1.228704
## dtype: float64
```

```
print(results_B.mse_resid)
```

```
## 0.9637630583005254
```

We see that the estimations of the coefficients are quite good, but for the variance of the residuals it is rather off. The results with a different seed and $N = 10$ are below:

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm

df = generate_dataset(N=10, seed=3)
df['intercept'] = 1

model = sm.OLS(df['Y'], df[['intercept', 'X1']])
results_A = model.fit()
print('Model A')
```

```
## Model A
```

```
print(results_A.params)
```

```
## intercept   -5.337987
```

```
## X1           4.261387
## dtype: float64
```

```
print(results_A.mse_resid)
```

```
## 2.2986972619872503
```

```
model = sm.OLS(df['Y'], df[['intercept', 'X2']])
results_B = model.fit()
print('Model B')
```

```
## Model B
```

```
print(results_B.params)
```

```
## intercept   -5.212091
## X2           1.203003
## dtype: float64
```

```
print(results_B.mse_resid)
```

```
## 0.6373652295986479
```

As expected, the estimations deteriorate. One could also check the variances of the estimations to show that for $N = 10$ it is larger than for $N = 1000$.

Grading: 1 point for results when $N \to \infty$. 1 point for estimations with code.

(c) Let us now consider the full model

$$Y_i = \gamma_2 X_{2,i} + \gamma_1 X_{1,i} + \gamma_0 + \varepsilon_i$$

where $i \in \{1, \ldots, n\}$ and the ε_i are independent $\mathcal{N}(0, \sigma^2)$ random variables. For the previously simulated data with $n = 10$, estimate $\hat{\gamma}_0, \hat{\gamma}_1, \hat{\gamma}_2, \hat{\sigma}^2$ and compare them with the parameters obtained in item (b). What can you say about the effects of X_1 and X_2 on Y? And about their correlation? **(1.0 points)**

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import statsmodels.api as sm

df = generate_dataset(N=10, seed=3)
df['intercept'] = 1

model = sm.OLS(df['Y'], df[['intercept', 'X1', 'X2']])
results_C = model.fit()
print('Model C')
```

```
## Model C
```

```
print(results_C.params)
```

```
## intercept   -5.213516
## X1           0.101043
## X2           1.177069
## dtype: float64
```

```
print(results_C.mse_resid)
```

```
## 0.7274008900209381
```

```
print(results_C.summary())
```

```
##                            OLS Regression Results
## ==============================================================================
## Dep. Variable:                      Y   R-squared:                       0.967
## Model:                            OLS   Adj. R-squared:                  0.958
## Method:                 Least Squares   F-statistic:                     102.8
## Date:                Thu, 15 May 2025   Prob (F-statistic):           6.47e-06
## Time:                        13:21:24   Log-Likelihood:                -10.815
## No. Observations:                  10   AIC:                             27.63
## Df Residuals:                       7   BIC:                             28.54
## Df Model:                           2
## Covariance Type:            nonrobust
## ==============================================================================
##                  coef    std err          t      P>|t|      [0.025      0.975]
## ------------------------------------------------------------------------------
## intercept     -5.2135      0.275    -18.974      0.000      -5.863      -4.564
## X1             0.1010      1.022      0.099      0.924      -2.315       2.517
## X2             1.1771      0.275      4.276      0.004       0.526       1.828
## ==============================================================================
## Omnibus:                        3.201   Durbin-Watson:                   2.073
## Prob(Omnibus):                  0.202   Jarque-Bera (JB):                1.410
## Skew:                           0.919   Prob(JB):                        0.494
## Kurtosis:                       2.943   Cond. No.                         13.2
## ==============================================================================
##
## Notes:
## [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
##
## /opt/homebrew/Caskroom/miniforge/base/envs/isla2025/lib/python3.11/site-packages/scipy/stats/_
##   return hypotest_fun_in(*args, **kwds)
```

We see that the results with $N = 10$ are quite bad just as before. Furthermore, when we do a statistical inference on the coefficients, we see that the null hypothesis is not rejected for γ_1, giving the impression that it has no influence on Y, when in fact it does (by construction). This effect is mainly due to the correlation between X_1 and X_2 and the lack of available observations, which leads to very poor estimators.

Grading: 0.25 points for the estimation of parameters, 0.75 points for the statistical inference and discussion regarding correlation between variables.

▶ Part 3: Adaboost (5.0 points)

Recall that the AdaBoost algorithm goes as follows:

Given: $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_m, y_m)$ where $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{-1, +1\}$

Initialize: $D_1(i) = 1/m$ for $i = 1, \ldots, m$

For $t = 1, \ldots, T$:

- Train a weak learner h_t to minimize the weighted training error:

$$\varepsilon_t = \sum_{i=1}^{m} D_t(i) \mathbf{1}\Big(h_t(\mathbf{x}_i) \neq y_i\Big) \tag{1}$$

- Choose the step-size:

$$\alpha_t = \frac{1}{2} \log\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right) \tag{2}$$

- Update, for $i = 1, \ldots, m$:

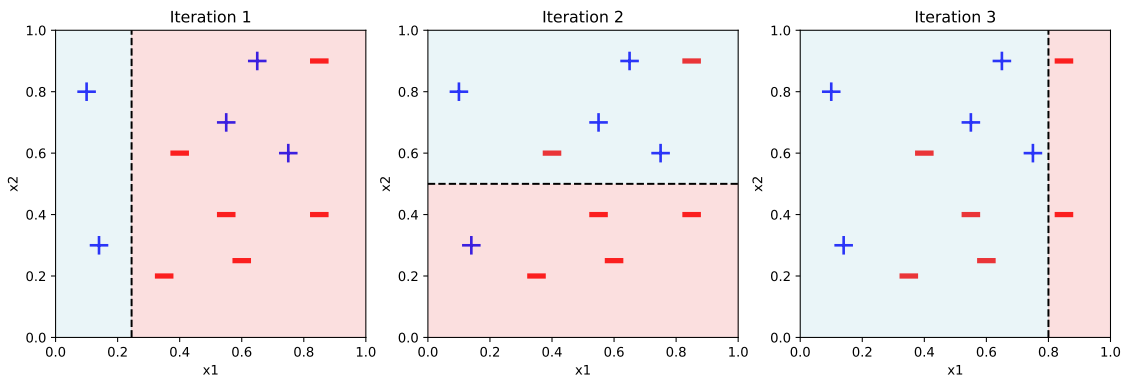
$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} \times \begin{cases} e^{-\alpha_t} & \text{if } h_t(\mathbf{x}_i) = y_i \\ e^{+\alpha_t} & \text{if } h_t(\mathbf{x}_i) \neq y_i \end{cases} = \frac{D_t(i) \exp(-\alpha_t y_i h_t(\mathbf{x}_i))}{Z_t} \tag{3}$$

  where Z_t is a normalization factor chosen so that D_{t+1} is a probability distribution.

  Output the final hypothesis as:

$$H(\mathbf{x}) = \text{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(\mathbf{x})\right) \tag{4}$$

The graphs below show three iterations of Adaboost using a depth 1 decision tree (i.e. a decision stump) on a given dataset. Each dashed line represents the decision boundary of h_t at iteration $t \in \{1, 2, 3\}$, and the shaded regions represent the predictions, positive (+1, blue) or negative (-1, red).



(a) For each iteration in the figure above, find the weighted training error ε_t and importance α_t of h_t according to the Adaboost algorithm described above. For $t = 2$ and $t = 3$ find the weight normalization Z_t and record the updated weight for each point. **(1.0 point)**

The easiest way to answer this question would be to implement a function that calculates the errors and weights automatically, instead of doing everything by hand.

```
w_1 = np.ones(11)
Z_1 = np.sum(w_1)
w_1 = w_1 / Z_1

y_true = np.array([+1, +1, -1, -1, -1, -1, +1, +1, +1, -1, -1])
y_pred_1 = np.array([+1, +1, -1, -1, -1, -1, -1, -1, -1, -1, -1])
```

```
e_1 = np.dot(w_1, y_pred_1 != y_true)
alpha_1 = 1/2 * np.log((1 - e_1)/e_1)

print("e_1:", e_1)
```

```
## e_1: 0.2727272727272727
```

```
print("alpha_1:", alpha_1)
```

```
## alpha_1: 0.49041462650586315
```

```
print("Z_1", Z_1)
```

```
## Z_1 11.0
```

```
w_2 = np.multiply(w_1, np.exp(-alpha_1 * np.multiply(y_true, y_pred_1)))
Z_2 = np.sum(w_2)
w_2 = w_2 / Z_2

y_true = np.array([+1, +1, -1, -1, -1, -1, +1, +1, +1, -1, -1])
y_pred_2 = np.array([+1, -1, -1, +1, -1, -1, +1, +1, +1, +1, -1])
e_2 = np.dot(w_2, y_pred_2 != y_true)
alpha_2 = 1/2 * np.log((1 - e_2)/e_2)

print("e_2:", e_2)
```

```
## e_2: 0.18750000000000006
```

```
print("alpha_2:", alpha_2)
```

```
## alpha_2: 0.7331685343967134
```

```
print("Z_2", Z_2)
```

```
## Z_2 0.8907235428302465
```

```
w_3 = np.multiply(w_2, np.exp(-alpha_2 * np.multiply(y_true, y_pred_2)))
Z_3 = np.sum(w_3)
w_3 = w_3 / Z_3

y_true = np.array([+1, +1, -1, -1, -1, -1, +1, +1, +1, -1, -1])
y_pred = np.array([+1, +1, +1, +1, +1, +1, +1, +1, +1, -1, -1])
e_3 = np.dot(w_3, y_pred != y_true)
alpha_3 = 1/2 * np.log((1 - e_3)/e_3)

print("e_3:", e_3)
```

```
## e_3: 0.28205128205128205
```

```
print("alpha_3:", alpha_3)
```

```
## alpha_3: 0.4671546186884167
```

```
print("Z_3", Z_3)
```

```
## Z_3 0.7806247497998
```

(b) Load the `datasets/dataset_ex3.csv` file into a dataframe (hint: `pandas.read_csv` is your friend) and make a scatter plot of the data points using the same markers and colors as in the figure above (hint: you can use option `marker='+'` and `marker='_'` in the `ax.scatter` function from `matplotlib`). **(1.0 point)**

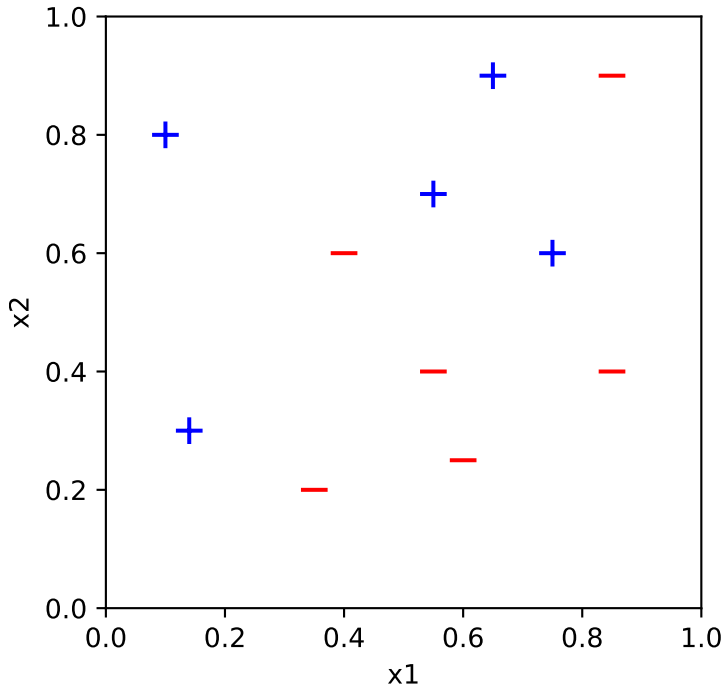
```
df = pd.read_csv('datasets/dataset_ex3.csv')
X = df[['x1', 'x2']].values
y = df['label']
y = np.where(y == 'plus', 1, -1)  # Convert labels to binary

fig, ax = plt.subplots(figsize=(4, 4))
idx_plus = np.where(y == 1)[0]
ax.scatter(X[idx_plus, 0], X[idx_plus, 1], c='b', marker='+', s=100)
idx_minus = np.where(y == -1)[0]
ax.scatter(X[idx_minus, 0], X[idx_minus, 1], c='r', marker='_', s=100)
ax.set_xlim(0, 1)
```

```
## (0.0, 1.0)
```

```
ax.set_ylim(0, 1)
```

```
## (0.0, 1.0)
```

```
ax.set_xlabel('x1')
ax.set_ylabel('x2')
fig.show()
```



(c) With the help of the `DecisionTreeClassifier` class from `sklearn`, fit a decision stump h_1 on the dataset corresponding to the first iteration of Adaboost. Remember to set an uniform weight w_1 over all data points in this first iteration and to ensure that the labels $y_i \in \{-1, +1\}$. Show that your predictions match those from the first iteration in the figure above. **(1.0 point)**

```
from sklearn.tree import DecisionTreeClassifier
w_1 = np.ones(len(y))
Z_1 = np.sum(w_1)
w_1 = w_1 / Z_1
clf_1 = DecisionTreeClassifier(max_depth=1)
_ = clf_1.fit(X, y, sample_weight=w_1)
y_pred_1 = clf_1.predict(X)
print(y_pred_1)
```

```
## [ 1  1 -1 -1 -1 -1 -1 -1 -1 -1 -1]
```

(d) Calculate the new weights w_2 of the data points based on predictions from iteration $t = 1$. Use them to fit a new decision stump h_2 on the weighted dataset (hint: every estimator in `sklearn` comes with an optional argument `sample_weight` in its `fit` method that can enforce weights on the loss function). Show that your predictions match those from the second iteration in the figure above. **(0.5 point)**

```
e_1 = np.dot(w_1, y_pred_1 != y)
alpha_1 = 1/2 * np.log((1 - e_1)/e_1)

w_2 = np.multiply(w_1, np.exp(-alpha_1 * np.multiply(y, y_pred_1)))
Z_2 = np.sum(w_2)
w_2 = w_2 / Z_2

clf_2 = DecisionTreeClassifier(max_depth=1)
_ = clf_2.fit(X, y, sample_weight=w_2)
y_pred_2 = clf_2.predict(X)
print(y_pred_2)
```

```
## [ 1 -1  1  1  1 -1 -1  1  1 -1 -1]
```

(e) Calculate the new weights w_3 of the data points based on predictions from iteration $t = 2$. Use them to fit a new decision stump h_3 on the weighted dataset. Show that your predictions match those from the third iteration in the figure above. (**0.5 point**)

```
e_2 = np.dot(w_2, y_pred_2 != y)
alpha_2 = 1/2 * np.log((1 - e_2)/e_2)

w_3 = np.multiply(w_2, np.exp(-alpha_2 * np.multiply(y, y_pred_2)))
Z_3 = np.sum(w_3)
w_3 = w_3 / Z_3

clf_3 = DecisionTreeClassifier(max_depth=1)
_ = clf_3.fit(X, y, sample_weight=w_3)
y_pred_3 = clf_3.predict(X)
print(y_pred_3)
```

```
## [ 1  1  1  1  1  1  1  1 -1 -1  1]
```

```
e_3 = np.dot(w_3, y_pred_1 != y)
alpha_3 = 1/2 * np.log((1 - e_3)/e_3)
```

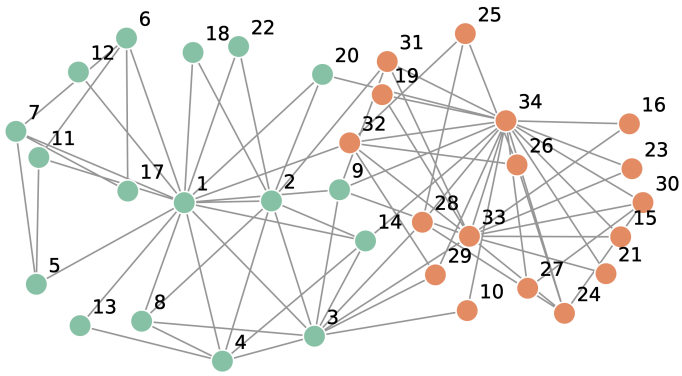
(f) Calculate the training error of the final ensemble classifier, i.e. the classifier that for each datapoint $\mathbf{x}$ outputs predictions $h(\mathbf{x}) = \text{sign}(\alpha_1 h_1(\mathbf{x}) + \alpha_2 h_2(\mathbf{x}) + \alpha_3 h_3(\mathbf{x}))$ **(1.0 point)**

```
y_1 = clf_1.predict(X)
y_2 = clf_2.predict(X)
y_3 = clf_3.predict(X)
y_pred = np.sign(alpha_1 * y_1 + alpha_2 * y_2 + alpha_3 * y_3)
print(np.mean(y_pred != y))
```

```
## 0.09090909090909091
```

▶ Part 4: Community detection (4.0 points)

We will consider a famous example of a social network from the network science literature called Wayne Zachary's "karate club" network. This network represents the pattern of friendships between members of a karate club at a US university, as determined by direct observation of the club's members over an extended period. This network is interesting because during the period of observation a dispute arose among the members of the club over whether to raise the club's fees and as a result the club eventually split into two equal parts with 17 members, the second group departing to form their own club. It is the two factions in this split, as reported by Zachary, that will form the ground truth for us when applying different algorithms for community detection. The figure below portrays the nodes of the graph (starting at 1) with their corresponding factions as colors.



First of all, you will have to load two `csv` files as follows with the help of `numpy`:

- `A = np.loadtxt('datasets/karate_adjacency.csv')` which gives the adjancecy matrix of the graph
- `F = np.loadtxt('datasets/karate_factions.csv')` which tells us the ground truth factions

Note that some questions below can be easily answered if you've read the paper "Modularity and community structure in networks" (available to you at `/article-newman.pdf`)

(a) Define in your own words the notion of modularity of a network and how it can be used to split a network into communities. Your description should include whether the modularity depends solely on the structure of the network or not. **(0.5 points)**

(b) Using the equations seen in class, calculate the modularity matrix of the graph. **(0.5 points)**

```
A = np.loadtxt('./datasets/karate_adjacency.csv')
F = np.loadtxt('./datasets/karate_factions.csv')
D = np.sum(A, axis=1)
m = np.sum(A) / 2
B = A - np.outer(D, D) / (2 * m)
print(B)
```

```
## [[-1.64102564  0.07692308 -0.02564103 ...  0.38461538 -1.23076923
##   -1.74358974]
##  [ 0.07692308 -0.51923077  0.42307692 ... -0.34615385 -0.69230769
##   -0.98076923]
##  [-0.02564103  0.42307692 -0.64102564 ... -0.38461538  0.23076923
##   -1.08974359]
##  ...
##  [ 0.38461538 -0.34615385 -0.38461538 ... -0.23076923  0.53846154
##    0.34615385]
##  [-1.23076923 -0.69230769  0.23076923 ...  0.53846154 -0.92307692
##   -0.30769231]
##  [-1.74358974 -0.98076923 -1.08974359 ...  0.34615385 -0.30769231
##   -1.8525641 ]]
```

(c) Once again using equations seen in class, calculate the modularity of the graph Q when fixing the classes of each node according to their ground truth factions as given in array `F`. (Hint: be sure to encode the factions with -1 and +1 as discussed in class) **(1.0 points)**

```
S = 2 * (F - 1) - 1
Q = S.T @ B @ S
print(Q)
```

```
## 111.76923076923077
```

(d) Based on the eigenvalue and eigenvector decomposition of the modularity matrix, split the graph into two communities. Interpret the magnitude of the coordinates of the leading eigenvector (i.e. the one related to the largest eigenvalue) and explain how it can be related to the "faction ambiguity" of each vertex in the graph. Are there any nodes for which the split look more ambiguous than others? Relate your answer with the figure of the graph above **(1.5 points)**

```
w, v = np.linalg.eig(B)
idx = np.argsort(w)[::-1]
w = w[idx]
v = v[:, idx]
v1 = np.real(v[:, 0])
S1 = -1*np.sign(v1)
```

There are certain nodes for which the corresponding eigenvalue is close to zero, indicating ambiguity in the choice of the faction. This happens mostly with nodes that are in the frontier between the factions.

(e) What would happen if all the eigenvalues of the modularity matrix were smaller than zero? What would this indicate in terms of the structure of the network? **(0.5 points)**

That there's no viable split of the graph in terms of modularity.